

# Machine Learning

**Instructor: Hafiz Abdul Rehman**

**Lecture 19: Perceptron**

# McCulloch-Pitts Neuron

(Akshay L Chandra: <https://towardsdatascience.com/mcculloch-pitts-model-5fdf65ac5dd1>)

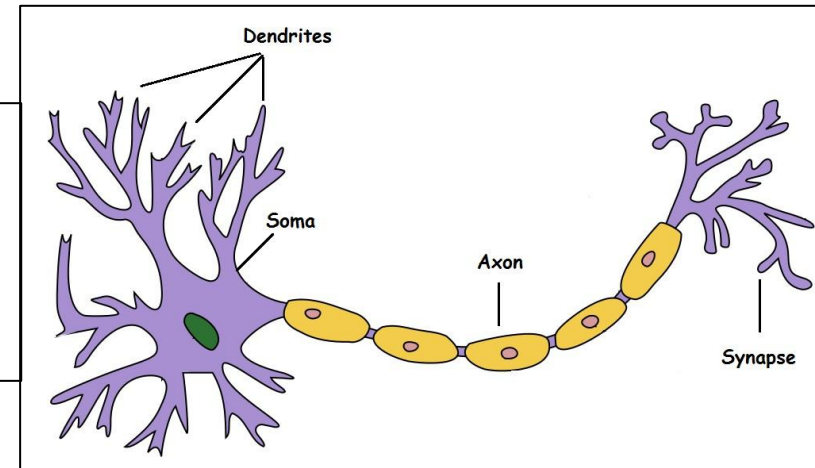
- The fundamental unit of ANNs – an artificial neuron
- 1943 by McCulloch and Pitts: Mimicking the functionality of a biological neuron

**Dendrites** receive signals from other neurons

**Soma** processes the information

**Axons** transmit the output

**Synapses** are the connections to other neurons

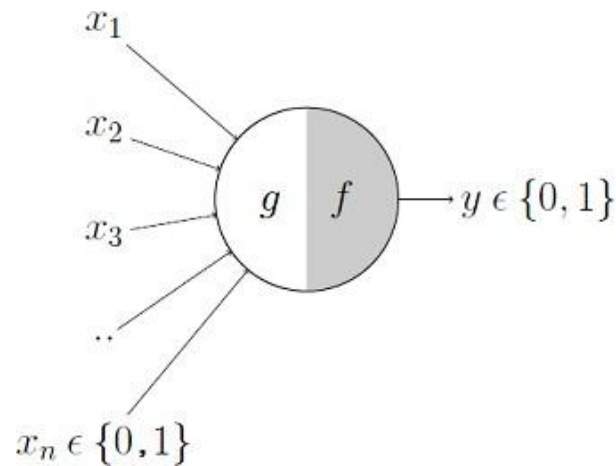


- About 86 billion of these in our brains on average!
- Each neuron gets activated/fired when its firing criteria is met
  - – Based on the aggregation of signals from the inputs

# McCulloch-Pitts Neuron

(Akshay L Chandra: <https://towardsdatascience.com/mcculloch-pitts-model-5fdf65ac5dd1>)

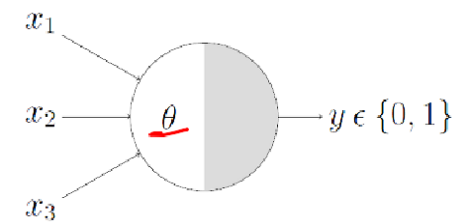
- The first computational model of a neuron was proposed by Warren McCulloch (neuroscientist) and Walter Pitts (logician) in 1943.



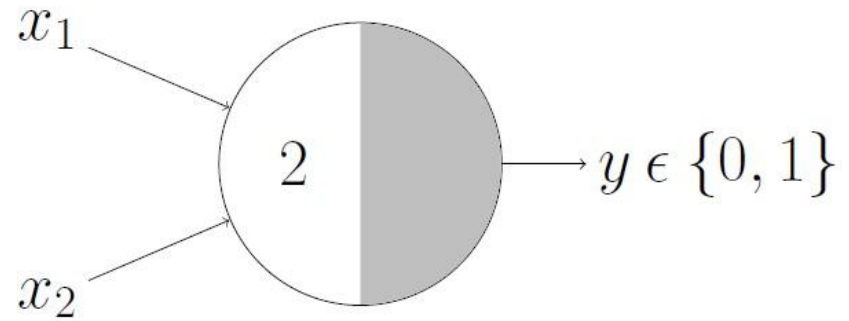
$$g(x_1, x_2, x_3, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \\ 0 & \text{if } g(\mathbf{x}) < \theta \end{cases}$$

- $g$  aggregates inputs,  $f$  makes decisions
- $\theta$  is a hand-coded threshold
- All  $x_i$  are binary inputs and  $y_i$  is a binary output

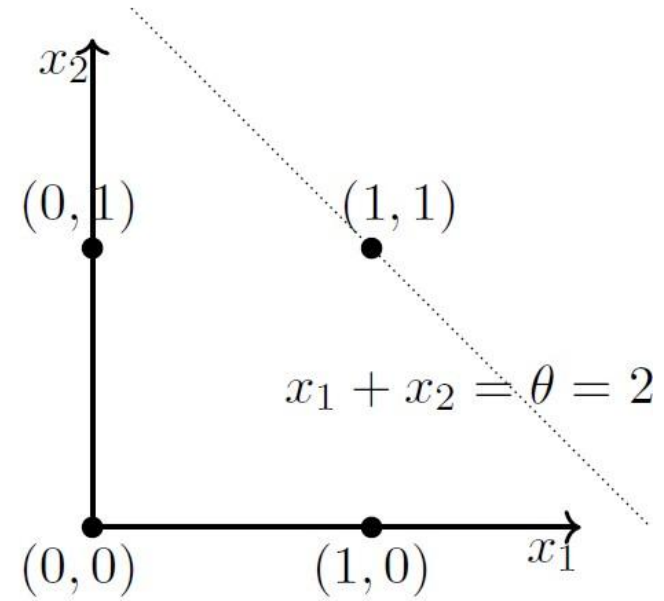


# Boolean Functions

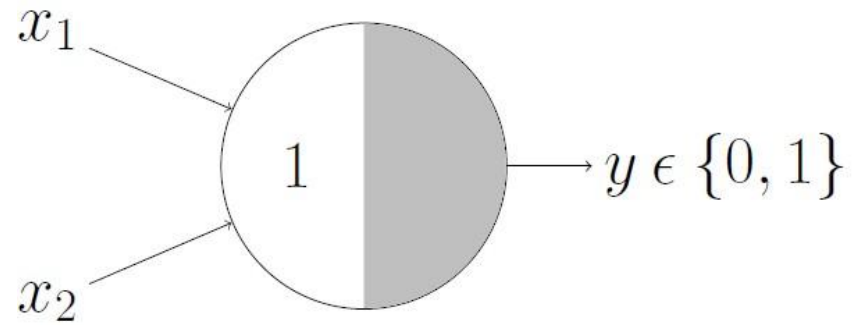


*AND function*

$$x_1 + x_2 = \sum_{i=1}^2 x_i \geq 2$$

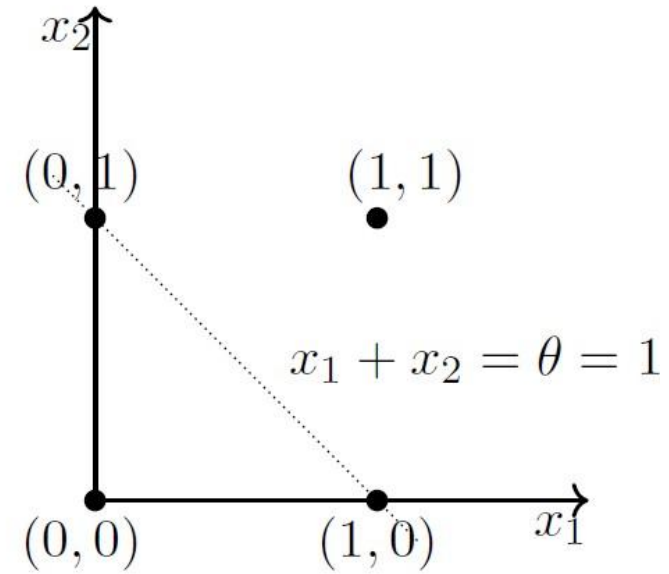


# Boolean Functions



*OR function*

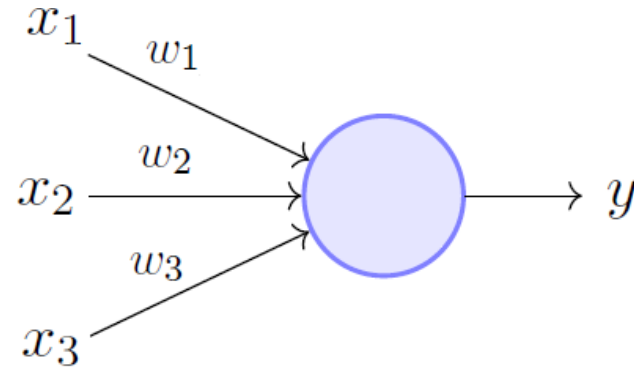
$$x_1 + x_2 = \sum_{i=1}^2 x_i \geq 1$$



# Limitations Of M-P Neuron

- Limited to Boolean inputs
- Hand-coded thresholds
- All inputs are equally important
  - – Are all inputs born equal? Is number of legs as important of a input in a human vs. cat classifier as number of ears?
- What about functions that are not linearly separable? E.g. the XOR function.
- In 1958, Frank Rosenblatt, an American psychologist, proposed the perception model
- Added weights and thresholds that could be learned. Allowed real-number inputs.

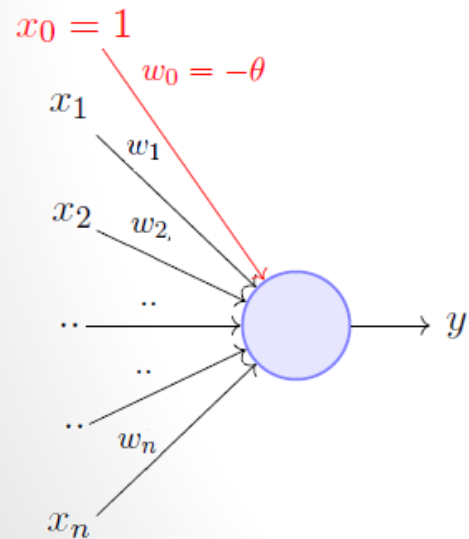
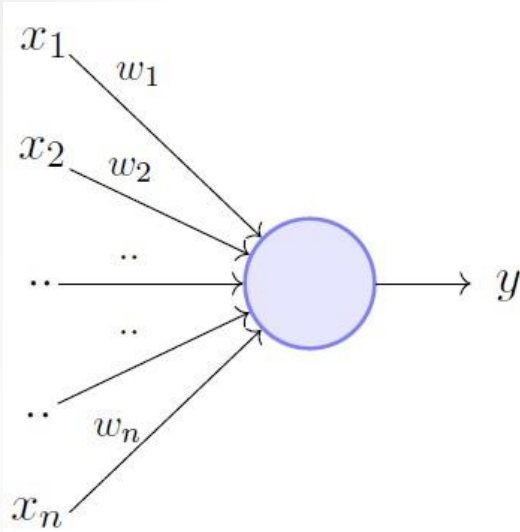
# The Perceptron



Perceptron Model (Minsky-Papert in 1969)

- Numerical weights associated with inputs
- No longer limited to Boolean inputs
- A learning mechanisms to train the weights and threshold

# The Perceptron



$$y = 1 \quad \text{if } \sum_{i=1}^n w_i * x_i \geq \theta$$

$$= 0 \quad \text{if } \sum_{i=1}^n w_i * x_i < \theta$$

Rewriting the above,

$$y = 1 \quad \text{if } \sum_{i=1}^n w_i * x_i - \theta \geq 0$$

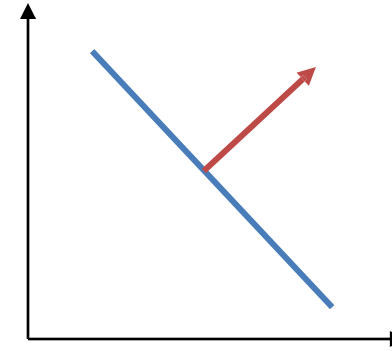
$$= 0 \quad \text{if } \sum_{i=1}^n w_i * x_i - \theta < 0$$

A more accepted convention,

$$y = 1 \quad \text{if } \sum_{i=0}^n w_i * x_i \geq 0$$

$$= 0 \quad \text{if } \sum_{i=0}^n w_i * x_i < 0$$

where,  $x_0 = 1$  and  $w_0 = -\theta$





# The OR function

| $x_1$ | $x_2$ | OR |                                     |
|-------|-------|----|-------------------------------------|
| 0     | 0     | 0  | $w_0 + \sum_{i=1}^2 w_i x_i < 0$    |
| 1     | 0     | 1  | $w_0 + \sum_{i=1}^2 w_i x_i \geq 0$ |
| 0     | 1     | 1  | $w_0 + \sum_{i=1}^2 w_i x_i \geq 0$ |
| 1     | 1     | 1  | $w_0 + \sum_{i=1}^2 w_i x_i \geq 0$ |

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

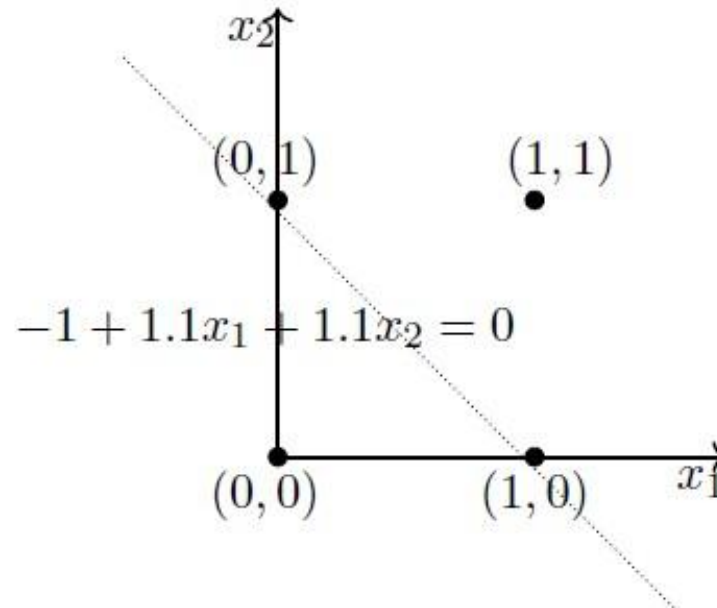
$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 > -w_0$$

One possible solution is

$$w_0 = -1, w_1 = 1.1, w_2 = 1.1$$



# The XOR function

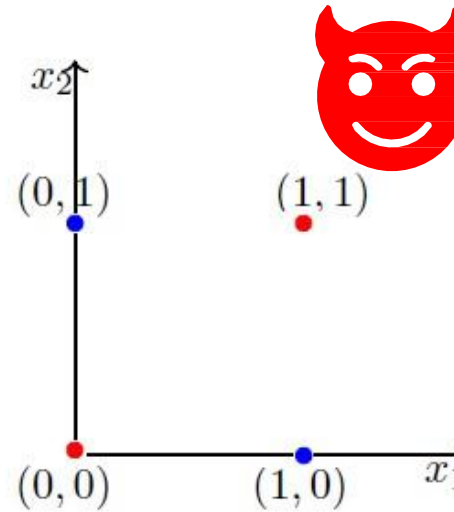
| $x_1$ | $x_2$ | XOR |                                     |
|-------|-------|-----|-------------------------------------|
| 0     | 0     | 0   | $w_0 + \sum_{i=1}^2 w_i x_i < 0$    |
| 1     | 0     | 1   | $w_0 + \sum_{i=1}^2 w_i x_i \geq 0$ |
| 0     | 1     | 1   | $w_0 + \sum_{i=1}^2 w_i x_i \geq 0$ |
| 1     | 1     | 0   | $w_0 + \sum_{i=1}^2 w_i x_i < 0$    |

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 < -w_0$$



- The fourth equation contradicts the second and the third equation
- There are no perceptron solutions for non-linearly separated data
- A single perceptron cannot learn to separate the data that are non-linear in nature
- In the book published by Minsky and Papert in 1969, the authors implied that, since a single artificial neuron is incapable of implementing some functions such as the XOR logical function, larger networks also have similar limitations, and therefore should be dropped.
- Later research on three-layered perceptions showed how to implement such functions, therefore saving the technique from obliteration.

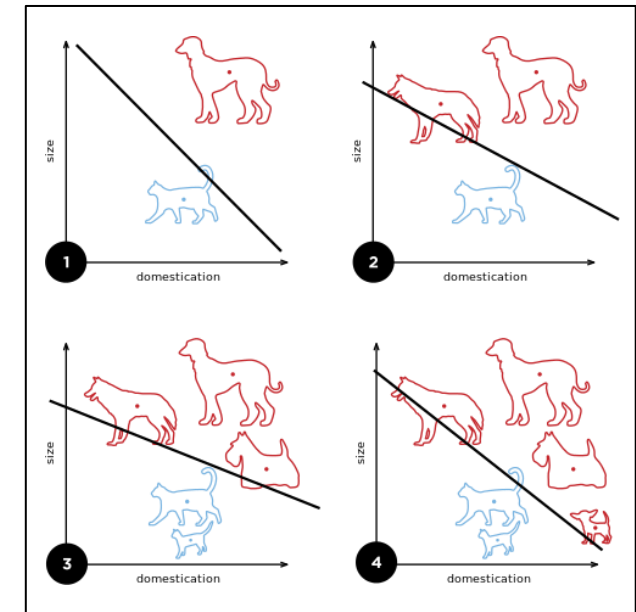
# The Perceptron

- A perceptron separates the input space into two halves, positive and negative.
- All the inputs that produce *true* lie on one side (positive half space) and all the inputs that produce *false* lie on the other side (negative half space)

A single perceptron can only be used to implement linearly separable functions

- Just like the M-P neuron

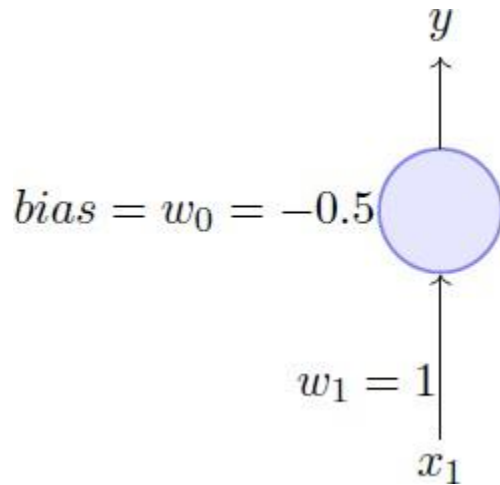
- The differences:
  - The inputs can be assigned different importance
  - The weights and the threshold can be learned
  - The inputs can be real values



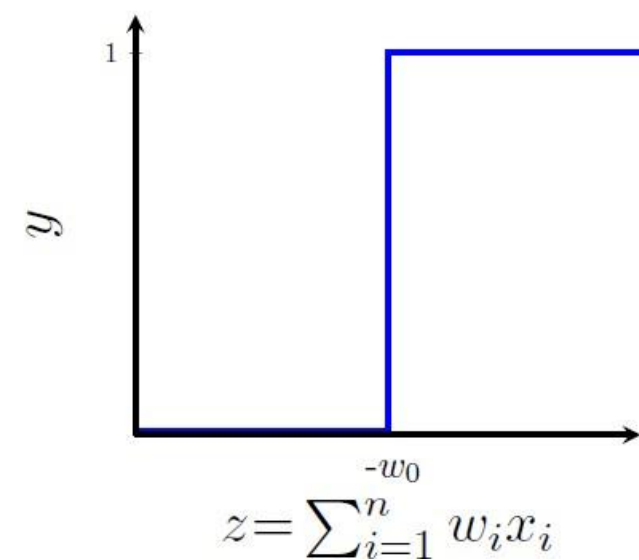
# Connecting the dots

(Akshay L Chandra: <https://towardsdatascience.com/perceptron-the-artificial-neuron-4d8c70d5cc8d>)

- The perceptron employs very harsh thresholds!
- E.g. if the threshold is 0.5: Input = 0.49 would yield a negative and input=0.51 would yield a positive output



$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$



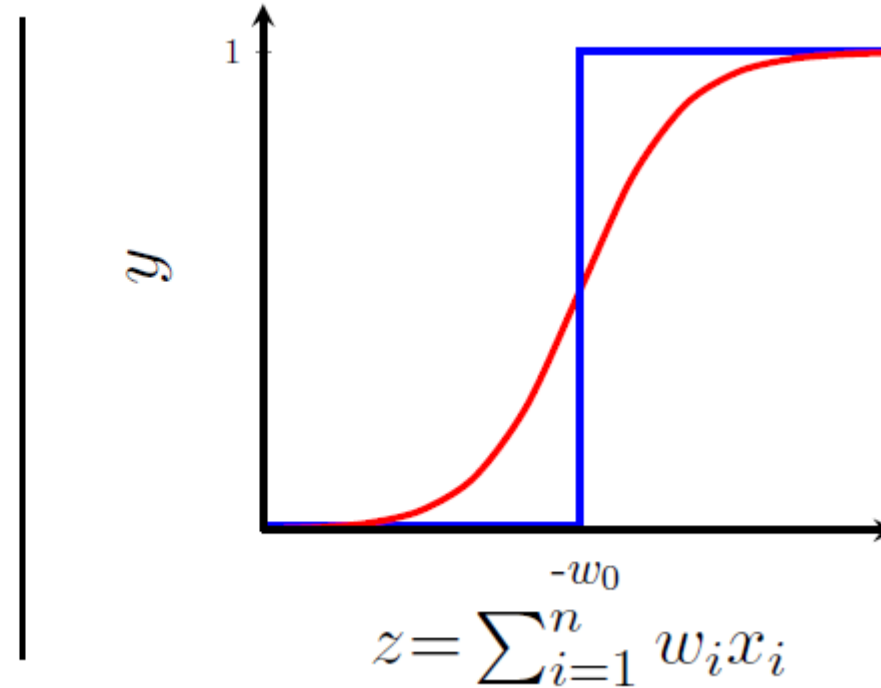
# Connecting the dots – Other Activation Functions

## Functions

(Akshay L Chandra: <https://towardsdatascience.com/perceptron-the-artificial-neuron-4d8c70d5cc8d>)

- Hence, the need for smoother functions like the sigmoid!

$$y = \frac{1}{1 + e^{-(w_0 + \sum_{i=1}^n w_i x_i)}}$$



- No more sharp transition around the  $w_0$
- The output is no longer binary but a real value between 0 and 1 which can be interpreted as a probability
- So instead of yes/no decision, we get the probability of yes.
- The output is **smooth**, **continuous** and **differentiable**.

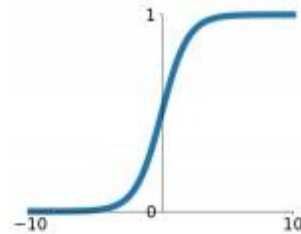


# Other Activation Functions ("squishifiers")

(Stanford: [http://cs231n.stanford.edu/slides/2017/cs231n\\_2017\\_lecture6.pdf](http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture6.pdf))

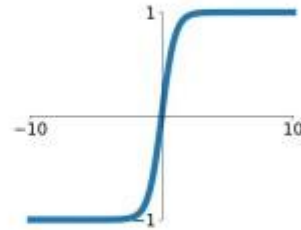
## Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



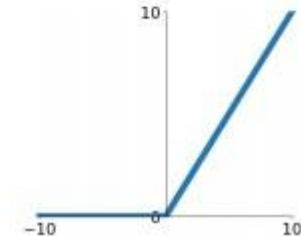
## tanh

$$\tanh(x)$$



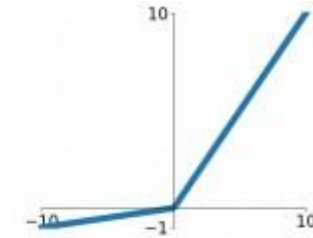
## ReLU

$$\max(0, x)$$



## Leaky ReLU

$$\max(0.1x, x)$$

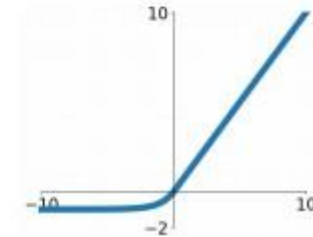


## Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

## ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



# Up next

- Perceptron Learning Algorithm
  - Geometric Intuition of why the learning works



# Perceptron Algorithm

(Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lectures 3-6,  
<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html>)

Initialize  $\vec{w} = 0$

while true do

$m = 0$

    for  $(x_i, y_i) \in D$  do

        if  $y_i(w^T \cdot x_i) \leq 0$  then

$\vec{w} \leftarrow \vec{w} + y_i \vec{x}_i$

$m \leftarrow m + 1$

        end if

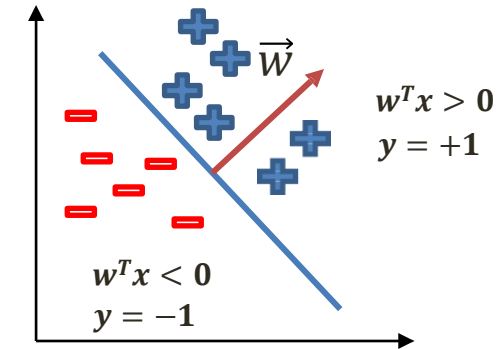
    end for

    if  $m = 0$  then

*break*

    end if

end while



Correct classification

$y_i = +1, w^T x_i > 0$

$y_i = -1, w^T x_i < 0$

Incorrect classification

$y_i = +1, w^T x_i < 0$

$y_i = -1, w^T x_i > 0$



# Up next

- Perceptron Convergence
  - Do we have any guarantee that the algorithm will converge given the data is linearly separable?



# Thank You